

Parte 3

Para cada uno de los ejercicios anteriores, resuelva los splines cúbicos de frontera natural para todos los valores de $x_2 \in \mathbb{R}$

Realice una animación de la variación de los splines cúbicos al variar x_2

```
In [2]: %matplotlib qt
import numpy as np
import sympy as sym
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

# =====
# CONFIGURACIÓN: Elige qué ejercicio deseas ejecutar (1, 2 o 3)
# =====
EJERCICIO = 2

# Configuración de datos según el ejercicio seleccionado
if EJERCICIO == 1:
    xs_fijas = [0, 1, 2]
    ys_dinamicas = [1, 5, 3]      # Tercer punto (y_2 = 3)
    y_limites = (-12, 10)        # Límites de visualización en Y
    y2_destino = -8               # Hasta dónde bajará el punto por sí solo
elif EJERCICIO == 2:
    xs_fijas = [0, 1, 2]
    ys_dinamicas = [-5, -4, 3]   # Tercer punto (y_2 = 3)
    y_limites = (-15, 8)
    y2_destino = -10
elif EJERCICIO == 3:
    xs_fijas = [0, 1, 2, 3]
    ys_dinamicas = [-1, 1, 5, 2] # Tercer punto (y_2 = 5)
    y_limites = (-10, 12)
    y2_destino = -6

# =====
# 1. Función Algorítmica de Splines Cúbicos Naturales
# =====
def cubic_spline(xs: list[float], ys: list[float]) -> list[sym.Symbol]:
    points = sorted(zip(xs, ys), key=lambda x: x[0])
    xs = [x for x, _ in points]
    ys = [y for _, y in points]
    n = len(points) - 1
    h = [xs[i + 1] - xs[i] for i in range(n)]

    alpha = [0] * (n + 1)
    for i in range(1, n):
        alpha[i] = 3 / h[i] * (ys[i + 1] - ys[i]) - 3 / h[i - 1] * (ys[i] - ys[i - 1])

    l = [1]
    u = [0]
    z = [0]

    for i in range(1, n):
        l += [2 * (xs[i + 1] - xs[i - 1]) - h[i - 1] * u[i - 1]]
        u += [h[i] / l[i]]
        z += [(alpha[i] - h[i - 1] * z[i - 1]) / l[i]]

    l.append(1)
    z.append(0)
```

```

c = [0] * (n + 1)

x = sym.Symbol("x")
splines = []
for j in range(n - 1, -1, -1):
    c[j] = z[j] - u[j] * c[j + 1]
    b = (ys[j + 1] - ys[j]) / h[j] - h[j] * (c[j + 1] + 2 * c[j]) / 3
    d = (c[j + 1] - c[j]) / (3 * h[j])
    a = ys[j]
    S = a + b * (x - xs[j]) + c[j] * (x - xs[j]) ** 2 + d * (x - xs[j]) ** 3
    splines.append(S)
splines.reverse()
return splines

# =====
# 2. Preparación de La Escena Gráfica de Matplotlib
# =====
fig, ax = plt.subplots(figsize=(9, 6))
ax.set_xlim(xs_fijas[0] - 0.5, xs_fijas[-1] + 0.5)
ax.set_ylim(y_limites)
ax.grid(True, linestyle="--", alpha=0.6)
ax.set_title(f"Ejercicio {EJERCICIO}: Haz CLIC en el punto VERDE ($y_2$) para que baje por sí

linea_spline, = ax.plot([], [], color="royalblue", lw=2.5, label="Spline Cúbico")
puntos_fijos, = ax.plot([], [], 'ro', markersize=8, label="Nodos Fijos")
punto_movil, = ax.plot([], [], 'go', markersize=10, label="Nodo Variable ($y_2$)")
ax.legend(loc="upper left")

# Generar la trayectoria del descenso automático desde la posición inicial
valores_descenso = np.linspace(ys_dinamicas[2], y2_destino, 120)
animacion = None
frame_actual = 0

# =====
# 3. Función de Renderizado (Frame por Frame)
# =====
def actualizar_cuadro(frame):
    global frame_actual

    # Modificar el valor de y_2 según el avance de la animación
    ys_dinamicas[2] = valores_descenso[frame]

    # Calcular nuevos splines simbólicos con la lógica original
    splines_simbolicos = cubic_spline(xs_fijas, ys_dinamicas)
    x_val = sym.Symbol("x")
    x_completo = []
    y_completo = []

    # Muestreo numérico de tramos suaves
    for j in range(len(xs_fijas) - 1):
        x_tramo = np.linspace(xs_fijas[j], xs_fijas[j+1], 50)
        f_num = sym.lambdify(x_val, splines_simbolicos[j], "numpy")
        y_tramo = f_num(x_tramo)
        x_completo.extend(x_tramo)
        y_completo.extend(y_tramo)

    # Volcar datos a los objetos gráficos
    linea_spline.set_data(x_completo, y_completo)

    # Separar nodos fijos del que se desplaza (índice 2)
    xf_fijos = [xs_fijas[i] for i in range(len(xs_fijas)) if i != 2]
    yf_fijos = [ys_dinamicas[i] for i in range(len(ys_dinamicas)) if i != 2]
    puntos_fijos.set_data(xf_fijos, yf_fijos)

    punto_movil.set_data([xs_fijas[2]], [ys_dinamicas[2]])

```

```

    frame_actual = frame
    return linea_spline, puntos_fijos, punto_movil

# Dibujar estado inicial
actualizar_cuadro(0)

# =====
# 4. Disparador del Clic
# =====
def al_dar_clic(event):
    global animacion
    if event.inaxes != ax: return

    # Validar proximidad al nodo x_2
    if abs(event.xdata - xs_fijas[2]) < 0.2:
        if animacion is None or frame_actual >= len(valores_descenso) - 1:
            ax.set_title(f"Ejercicio {EJERCICIO}: Animación automática en proceso...", fontsi
            # Disparar la animación cíclica/lineal sin requerir arrastre continuo
            animacion = FuncAnimation(fig, actualizar_cuadro, frames=len(valores_descenso),
                                     interval=25, repeat=False, blit=True)

            fig.canvas.draw_idle()

# Conexión del evento interactivo
fig.canvas.mpl_connect('button_press_event', al_dar_clic)

plt.show()

```